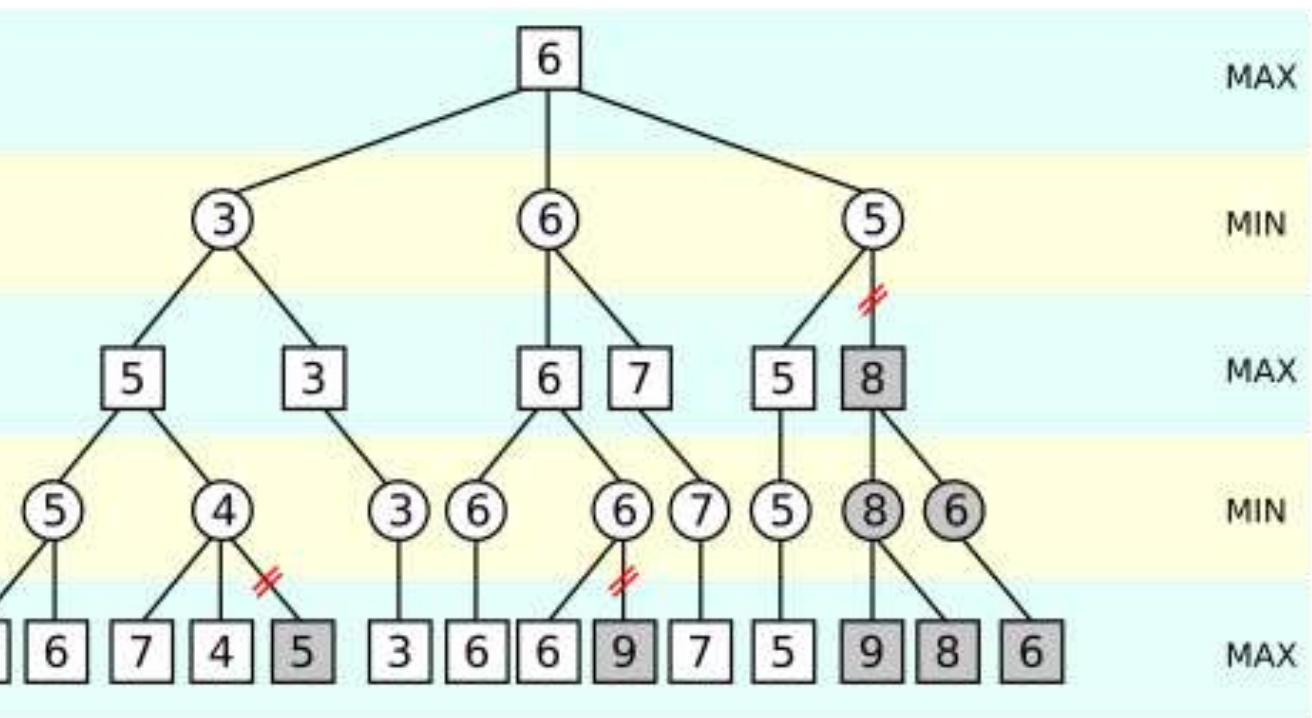


Introduction

- A chess "fortress" is a drawn position where one side holds an impenetrable defensive setup despite being materially behind
- Chess engines are tuned for tactical, materialistic play, but fortresses require long-term strategic judgement instead
- This makes fortress positions a rich, underexplored area for algorithmic study

Motivation

- Building an all-purpose chess engine in this timeframe, with one developer, would be impossible
- No current engine is designed exclusively for fortress positions
- Existing engines often misjudge fortresses, overvaluing material at the expense of positional understanding
- This leaves a niche that this project aims to fill



A diagram of the alpha-beta pruning algorithm

How do existing engines work?

Combination of heuristics and optimisation algorithms, especially alpha-beta pruning, then assigns an evaluation to the position

Heuristics include (but not limited to):

- Material advantage
- Piece activity
- King safety
- Tactics
- Positional advantage

Alpha-Beta Pruning:

Minimax algorithm that prunes away moves and lines that are worse than other lines from a given position

“Worse” as defined by the heuristics which can fail

How does my engine work?

Engine was composed of 3 parts:

- Exhaustive search
- Alpha-Beta pruning with rising evaluation
- Heuristic analysis

Stage 1: Exhaustive search

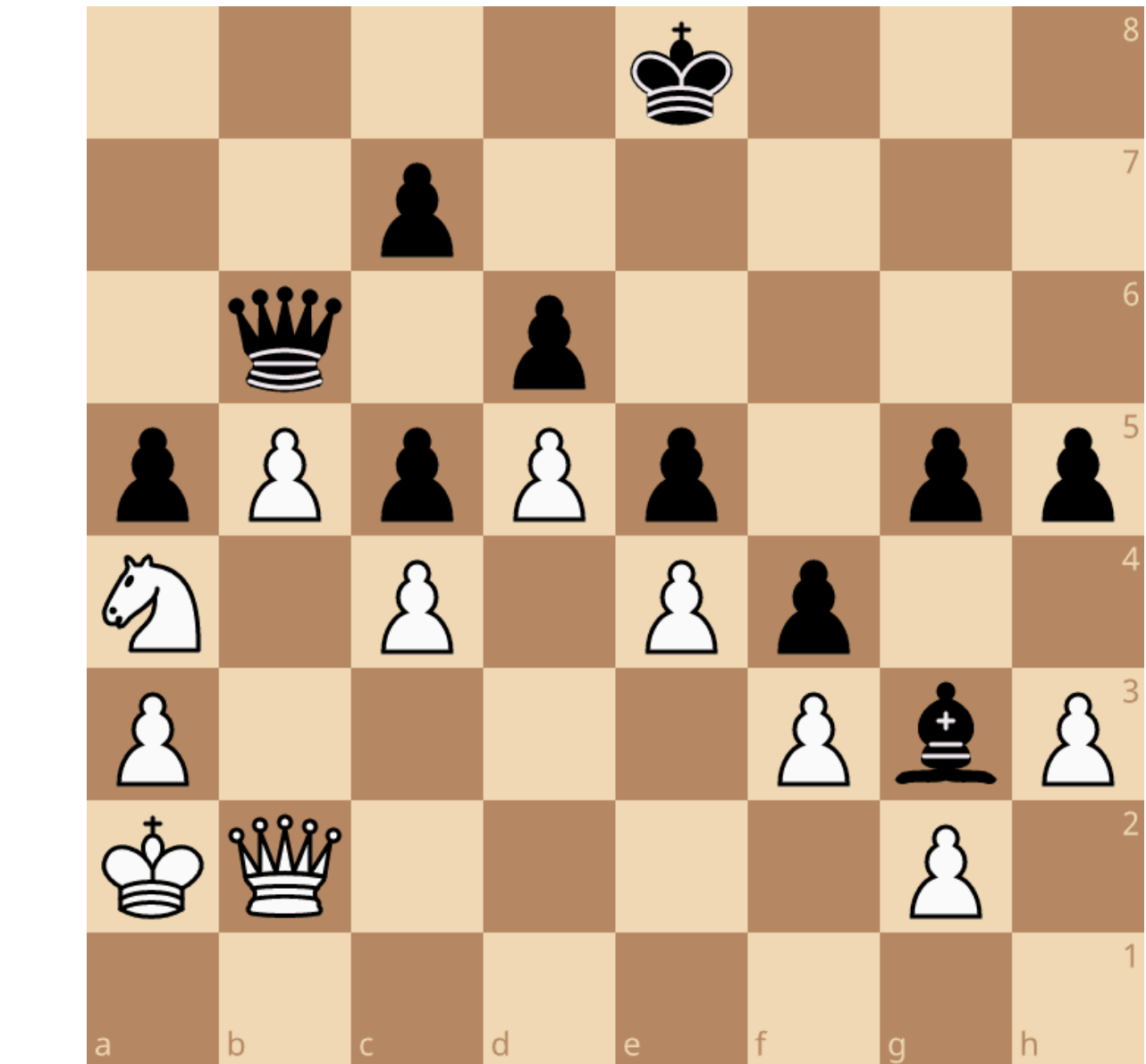
- Exhaustively searches through every possible move for 4-6 moves
- Ensures that no possible breakthrough move is missed, and that engine does not prune away less promising moves too early

Stage 2: Alpha-Beta pruning with rising evaluation

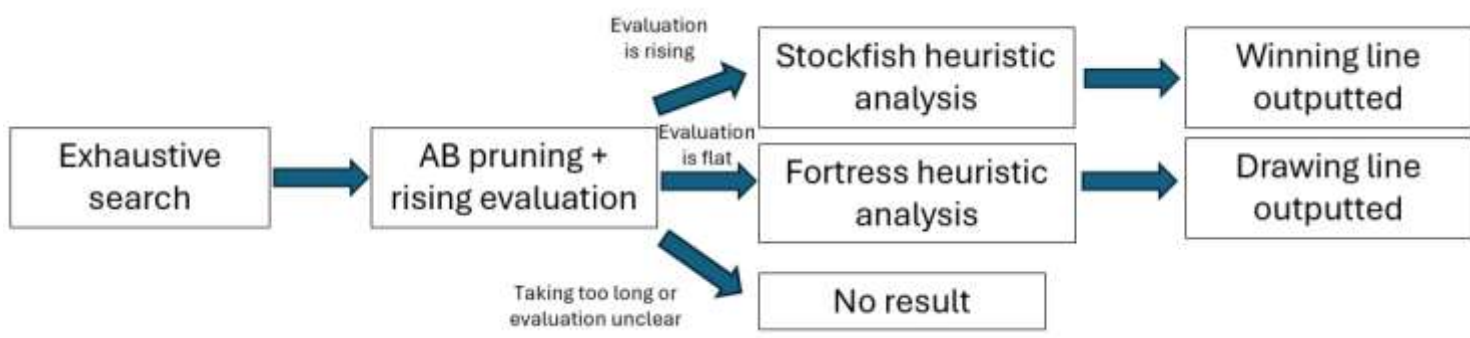
- Starts from every possible position found through the exhaustive search
- Runs a pruning search to find the most promising lines, and checks evaluation as the search goes deeper
- If evaluation stays flat – fortress is assumed
- If evaluation keeps rising – breakthrough assumed
- Examines these lines with the next stage to find winning lines

Stage 3: Heuristic analysis

- Analyses the resulting lines with heuristics
- If breakthrough move found: looks at things like material advantage, piece activity and tactics, tries to find a winning line, heuristics adapted from Stockfish
- If no breakthrough move found: falls back to fortress heuristics coded myself, holds the draw



Dvoretsky’s fortress study – most engines will take the free queen, but this leads to an indestructible fortress. Qd2 is the best move.



Flow chart of how the engine works.

How do current engines compare?

Current engine weaknesses:

- Prune away lines that might be winning, especially involving a sacrifice
- Are too materialistic:
- Will take free material even if this leads to a fortress
- Assigns an advantage to the side with more material even if the position is a dead draw

How my engine addresses this:

- Exhaustively checks every move to ensure possible sacrifices aren’t pruned away
- Rising evaluation phase prevents taking free material if there is a better line
- Reports either a win, draw, loss, or unclear if the position cannot be solved – to avoid hallucinating an advantage

Testing, Results and Evaluation

Testing:

Tested against tablebases, current engines and known fortress positions

Results:

Performed well in simpler positions
Struggled in more complex positions – either too slow or could not find the right line

Evaluation:

Engine underperformed expectations, but remains a solid proof of concept

Conclusions

- A solid first prototype, though performance fell short of initial goals
- Future work could include:
- Optimising the code for faster execution
- Incorporating neural networks (e.g. AlphaZero, Leela) for stronger play

Acknowledgements

- Lichess for providing a games database and engine tools
- Syzygy for their endgame tablebases
- Wikimedia commons for the alpha-beta pruning image